

COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #9: RISC v.s. CISC Architectures

2

RISC v.s. CISC Architectures

Why RISC Architecture?

Until 1995, vendors tended to design and build complex processors with rich ISA. Their goals were:

- To support high-level programming languages and OS
- To ease migration of software function to hardware
- To keep compatibility among successive generations of their processors

Such supports caused:

- Additional instructions to do complex functions
- Additional addressing modes

→ instruction formats of various lengths ☹

Such additions caused:

- Added logic levels and use of complex microprograms
- Need for higher integration density

Why RISC Architecture?

So, many processors with very complex instruction sets were designed and implemented.

Statistics showed that most of such complex instructions were never used.

This is because:

- Most compilers do not consider newly added complex instructions. So, programs written in high-level languages and compiled by older compilers will not use newly added instructions
 - Most assembly programmers use what they know from older machines and rarely use newly added instructions
- Why should we put such a huge complexity when we do not use them?!
- Worse: Why we should dissipate power for such added hardware when they are not used or rarely used?!

Why RISC Architecture?

The reasons for turning from CISC philosophy (with many complex instructions and large micro-codes) to RISC philosophy (having few, short and quick instructions, and no micro-codes):

- Bad effects of complex instructions → lower speed
 - ✓ CISC philosophy: use shorter instructions for more frequently used instructions
 - ✓ RISC philosophy: do not have instructions that are not frequently used

Why RISC Architecture?

The reasons for turning from CISC philosophy (with many complex instructions and large micro-codes) to RISC philosophy (having few, short and quick instructions, and no micro-codes):

- Bad effects of complex instructions → lower speed
- Better use of transistors in VLSI implementation
 - Goal was to implement the whole system on a chip.
 - 40-60% of chip area in CISC systems were used for control unit; this can be reduced to 10% in a RISC processor. The remaining 30-50% can be used to implement larger registers files, caches, parallelism, pipelining, etc.

Why RISC Architecture?

The reasons for turning from CISC philosophy (having many complex instructions and large micro codes) to RISC philosophy (having few, short and quick, and no micro-codes at all):

- Bad effects of complex instructions
- Better use of transistors in VLSI implementation
- Microprogramming overheads
 - Microprogramming were used widely in 70s, when core memory was the dominant main memory type and faster but expensive memories were used for micro memory. The different became less and less in the next years.

Why RISC Architecture?

The reasons for turning from CISC philosophy (having many complex instructions and large micro codes) to RISC philosophy (having few, short and quick, and no micro-codes at all):

- Bad effects of complex instructions
- Better use of transistors in VLSI implementation
- Microprogramming overheads
- Effective use of compilers
 - Most optimizing compilers tend to use less instructions. Having less instructions in RISC in addition to the larger register files are great helps to compilers.

Why RISC Architecture?

Common design goals:

- **Maximizing processor speed**
- **Minimizing design and production costs**

Design principles to achieve above goals:

- **Simplicity favors regularity**
 - Consistent instruction format
 - Same number of operands (two sources and one destination)
 - Easier to encode and handle in hardware
- **Make the common case fast**
 - A RISC processor have only simple, commonly used instructions
 - Hardware to decode and execute instructions can be simple, small, and fast
 - More complex instructions (that are less common) can be performed using multiple simple instructions

Properties of RISC Architectures

- Smaller instruction set, shorter and fixed-sized instruction formats, few addressing modes
 - Instructions accessing memory operands are only **load** and **store** and other instructions work with register file.
 - Instruction length is fixed and not longer than word size.
- Large register file and on-chip cache
- Hardwired (not microprogrammed) control unit is popular
- Employing pipelining, super-pipelining, superscalar techniques for higher throughput

As a result:

- Most instructions are executed in a cycle; in superscalar designs more than one instructions are executed in one CPU cycle
- Better supporting the compilers
 - higher performance of programs written with high-level programming languages

CISC v.s. RISC

Runtime of a program depends on three main parameters:

- Program length or #of instructions (L) $L_{RISC} > L_{CISC}$
- Number of clocks to execute one instruction (C) $C_{RISC} < C_{CISC}$
- Clock period (T) $T_{RISC} < T_{CISC}$

$$\text{Runtime} = L \times C \times T$$

Putting all together it is mostly RISC machines that result in faster execution of programs.

MIPS processor enjoys RISC architecture:

- Developed by John Hennessy and his colleagues at Stanford in the 1980's.
- Used in many commercial systems, including Silicon Graphics, Nintendo, Cisco, ...
- Used as a model for teaching Computer Organization/Structure/Architecture

We use it to teach control unit and pipelining techniques.

MIPS architecture:

- Operand's physical location in computer:
 - Register
 - Memory
 - Constant (also called *immediate*)
- MIPS has 32 32-bit registers
- Memory size is 2^{30} words (each of 32 bits), byte addressable (word address is always x4)
- MIPS called "32-bit architecture" because it operates on 32-bit data
- Registers:
 - \$ before name
 - Example: \$0 that shows "register zero"

Registers used for specific purposes:

- \$0 always holds the constant value 0.
- *saved registers*, \$s0-\$s7, used to hold variables (to be preserved by the subroutine)
- *temporary registers*, \$t0 - \$t9, used to hold intermediate values
- \$fp, points to the beginning of subroutine's stack space
- \$gp, points to the middle of a 64K block of main memory in the heap that holds constants and global variables.

Name	Register Number	Usage
\$zero	0	constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

Instruction formats (all 32 bits):

- **R-Type**: register operands
- **I-Type**: immediate operand
- **J-Type**: for jumping (discuss later)

Recall that **simplicity favors regularity!**
(in MIPS word length = instruction Length) ☺

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

J-Type

op	addr
6 bits	26 bits

MIPS OPCODES

28:26 31:29	000	001	010	011	100	101	110	111
000	R-type		j	jal	beq	bne		
001	addi	addiu	slti	sltiu	andi	ori	xori	lui
010								
011								
100	lb	lh		lw	lbu	lhu		
101	sb	sh		sw				
110								
111								

2:0 5:3	000	001	010	011	100	101	110	111
000	sll		srl	sra	slv		srlv	srav
001	jr	jalr						
010	mfhi		mflo					
011	mult	multu	div	divu				
100	add	addu	sub	subu	and	or	xor	nor
101			slt	sltu				
110								
111								

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- 3 register operands:
 - rs, rt: source registers
 - rd: destination register
- Other fields:
 - op: the *operation code* or *opcode* (0 for R-type instructions)
 - funct: the *function* with *op*, tells computer what operation to perform
 - shamt: the *shift amount* for shift instructions, otherwise it's 0

R-type Example:

Assembly Code

```
add $s0, $s1, $s2
sub $t0, $t3, $t5
```

Field Values

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34

6 bits 5 bits 5 bits 5 bits 5 bits 6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)

6 bits 5 bits 5 bits 5 bits 5 bits 6 bits

Note the order of registers in the assembly code is: `add rd, rs, rt`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

- 3 operands:
 - rs, rt: register operands
 - imm: 16-bit immediate (mostly treated as signed, but not always)
- Other fields:
 - op: the *opcode* determines the operation

I-type Example:

Assembly Code

```
addi $s0, $s1, 5
addi $t0, $s3, -12
lw    $t2, 32($0)
sw    $s1, 4($t1)
```

Field Values

op	rs	rt	imm
8	17	16	5
8	19	8	-12
35	0	10	32
43	9	17	4

6 bits

5 bits

5 bits

16 bits

Note the differing order of registers in assembly and machine codes:

```
addi rt, rs, imm
lw    rt, imm(rs)
sw    rt, imm(rs)
```

Machine Code

op	rs	rt	imm	
001000	10001	10000	0000 0000 0000 0101	(0x22300005)
001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
101011	01001	10001	0000 0000 0000 0100	(0xAD310004)

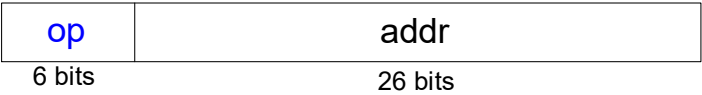
6 bits

5 bits

5 bits

16 bits

J-Type



- one operand:
 - addr: 26-bit address
- Used for jump instructions (j)

R-format Instructions Summary

add	rd, rs, rt	# $rd \leftarrow rs + rt$
sub	rd, rs, rt	# $rd \leftarrow rs - rt$
and	rd, rs, rt	# $rd \leftarrow rs \& rt$
or	rd, rs, rt	# $rd \leftarrow rs rt$
xor	rd, rs, rt	# $rd \leftarrow rs \wedge rt$
nor	rd, rs, rt	# $rd \leftarrow \sim(rs \wedge rt)$
slt	rd, rs, rt	# $rd \leftarrow rs < rt ? 1 : 0$
sltu	rd, rs, rt	# $rd \leftarrow rs < rt ? 1 : 0$
mult(u)	rs, rt	# $\{hi, lo\} \leftarrow rs \times rt$
div(u)	rs, rt	# $lo \leftarrow rs / rt, hi \leftarrow rs \% rt$
mfhi	rd	# $rd \leftarrow hi$
mflo	rd	# $rd \leftarrow lo$
jr	rs	# $PC \leftarrow rs$
jalr	rs	# $PC \leftarrow rs$ # $\$31(\$ra) \leftarrow PC + 4$

I-format Instructions Summary

addi	rt, rs, imm	# $rt \leftarrow rs + \text{sgn_xtnd}(imm)$
andi	rt, rs, imm	# $rt \leftarrow rs \& \text{zro_xtnd}(imm)$
ori	rt, rs, imm	# $rt \leftarrow rs \text{zro_xtnd}(imm)$
xori	rt, rs, imm	# $rt \leftarrow rs \wedge \text{zro_xtnd}(imm)$
lui	rt, imm	# $rt \leftarrow imm \ll 16$
slti	rt, rs, imm	# $rt \leftarrow rs < \text{sgn_xtnd}(imm) ? 1 : 0;$
sltiu	rt, rs, imm	# $rt \leftarrow rs < \text{zro_xtnd}(imm) ? 1 : 0;$
lw	rt, imm(rs)	# $rt \leftarrow \text{Mem}[rs + \text{sgn_xtnd}(imm)]$
sw	rt, imm(rs)	# $\text{Mem}[rs + \text{sgn_xtnd}(imm)] \leftarrow rt$
beq	rs, rt, imm	# if(rs = rt) then $PC \leftarrow PC + \text{sgn_xtnd}(imm) \times 4$
bne	rs, rt, imm	# if(rs $\neq$ rt) then $PC \leftarrow PC + \text{sgn_xtnd}(imm) \times 4$

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #9: RISC v.s. CISC Architectures23

J-format Instructions Summary

jimm26# PC ← {PC[31:28] | imm26 | 00}

jalimm26# \$31(\$ra) ← PC + 4;
PC ← {PC[31:28] | imm26 | 00}

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #9: RISC v.s. CISC Architectures24

MIPS Pseudo-instructions

Pseudo-instruction	MIPS Instructions
li \$s0,0x1234ABCD	lui \$s0, 0x1234 ori \$s0, 0xABCD
clear \$t0	add \$t0, \$0, \$0
move \$s1, \$s2	add \$s1, \$s2, \$0
nop	sll \$0, \$0, 0

12

The diagram illustrates a 32-bit RISC processor architecture. Key components and their interconnections are as follows:

- PC (Program Counter):** Provides the address to the **Instruction memory**.
- Instruction memory:** Outputs an **Instruction** to the **Registers** block.
- Registers:** A block containing four registers (Read register 1, Read register 2, Write register, Write data). It also includes a **Sign extend** unit that takes a 16-bit input and produces a 32-bit output.
- ALU (Arithmetic Logic Unit):** Receives inputs from the **Registers** (Read data 1, Read data 2) and the **Sign extend** unit. It also receives a **Shift left 2** control signal. The ALU output is connected to the **Zero** flag and the **Write data** input of the **Data memory**.
- Data memory:** Receives **Write data** from the ALU and outputs **Read data** to the **Registers** (Read data 1, Read data 2).
- Adders:** There are two adders. The first adder takes a 4-bit constant and the **PC** value as inputs. The second adder takes the output of the first adder and the **Read data 1** from the **Registers** as inputs. The output of the second adder is connected to the **PC** and the **Write data** input of the **Data memory**.
- Multiplexers (MUX):** There are two MUXes. The first MUX selects between the **Read data 1** from the **Registers** and the **Read data 2** from the **Registers** to be shifted left by 2. The second MUX selects between the **Read data 1** from the **Registers** and the **Read data 2** from the **Registers** to be written to the **Data memory**.

QUESTIONS?